PQC vs Classical Cryptography Report

Generated with Node.js Playwright using Python venv execution (`.venv/bin/python`).

Run count: 10

Critical Analysis: Massive Key-Size Difference

RSA-3072 public key

422 bytes

ML-KEM-768 public key

1184 bytes

RSA-OAEP ciphertext

384 bytes

ML-KEM ciphertext

1088 bytes

ML-KEM-768 public keys are approximately **2.81x** larger than RSA-3072 public keys in this PoC. ML-KEM ciphertexts are approximately **2.83x** larger than RSA-OAEP ciphertexts. This is the key engineering trade-off: post-quantum security generally requires larger transmitted artifacts, which has direct implications for storage, bandwidth, and protocol framing.

Shared secret verification: **Yes**

Performance Snapshot

RSA-3072 median keygen

195.359 ms

ML-KEM-768 median keygen

0.016 ms

Step-by-Step Script Behavior

1. The Node script validates that `.venv/bin/python`, `pqc_compare.py`, and the terminal screenshot file exist.
2. It executes the Python PoC with `--runs 10` and captures stdout.
3. It parses timing, public-key sizes, ciphertext sizes, and shared-secret match values from that output.
4. It computes the PQC-to-classical size ratios and injects those values into an HTML analysis report.
5. It embeds the project terminal screenshot and the raw Python terminal output for evidence and reproducibility.
6. Playwright renders the page, captures clear PNG screenshots (full report and key-size focus), then exports the PDF report.

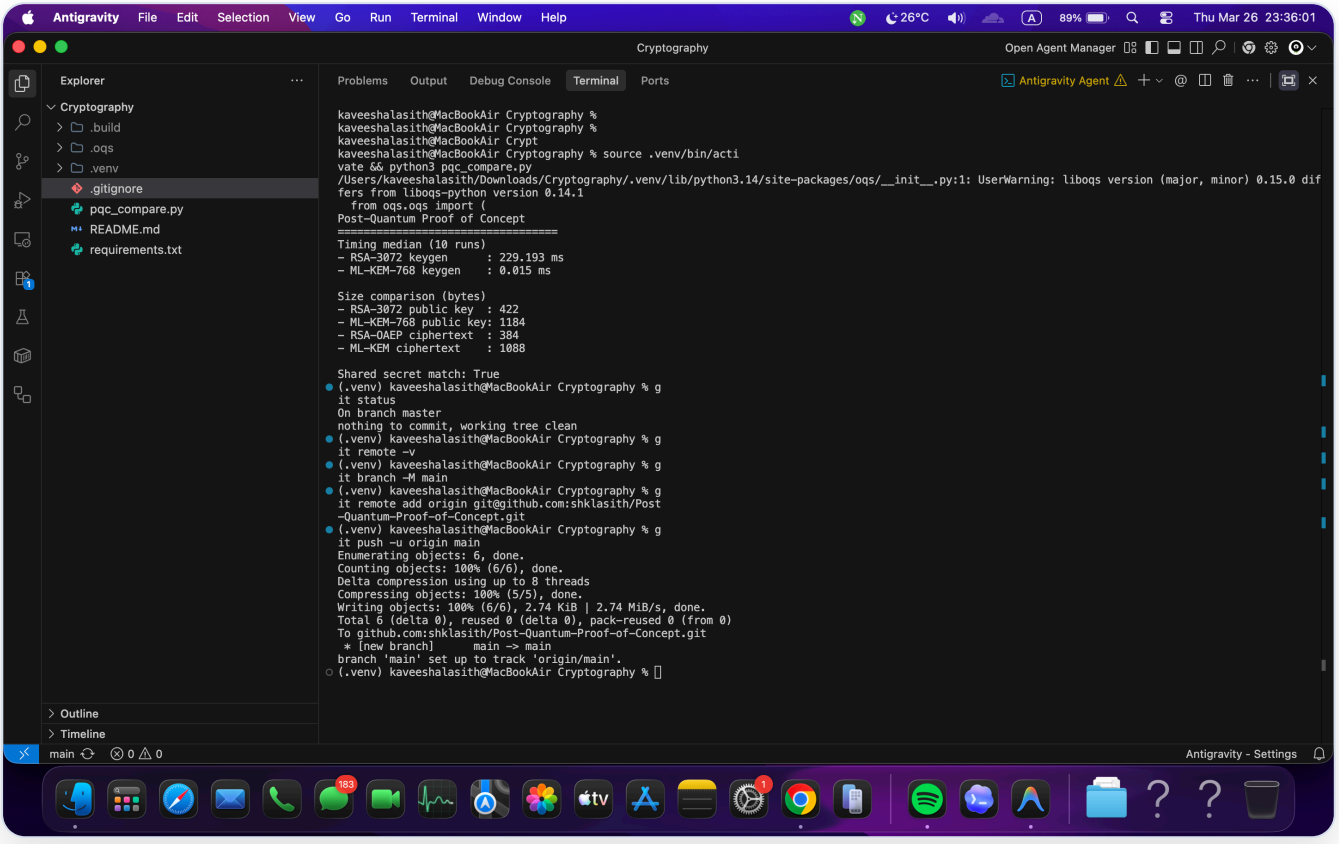
Terminal Output Evidence

```
Post-Quantum Proof of Concept
==================================
Timing median (10 runs)
- RSA-3072 keygen       : 195.359 ms
- ML-KEM-768 keygen     : 0.016 ms

Size comparison (bytes)
- RSA-3072 public key   : 422
- ML-KEM-768 public key: 1184
- RSA-OAEP ciphertext   : 384
- ML-KEM ciphertext     : 1088

Shared secret match: True
```

Screenshot captured from project folder:



Python Project Repository

https://github.com/shklasith/Post-Quantum-Proof-of-Concept.git